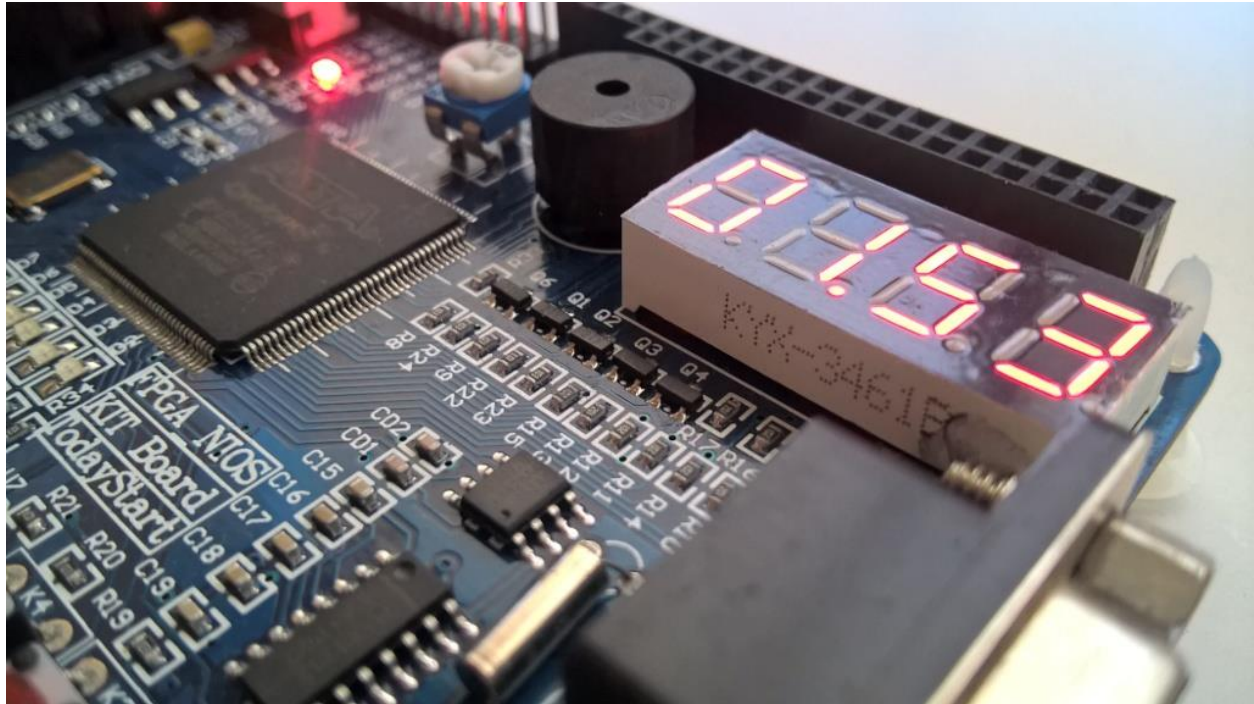


Proyecto: Reloj Digital

Date: 21 de junio de 2025



Professor:

Dr. Miguel Ángel Rivera Acosta

Estudiantes:

Alonso Emmanuel López Macias.

César Eduardo Inda Cenicerros.

Luis Alberto Castañeda Montaña.

José Antonio Garcia Madrigal.

Team: 9

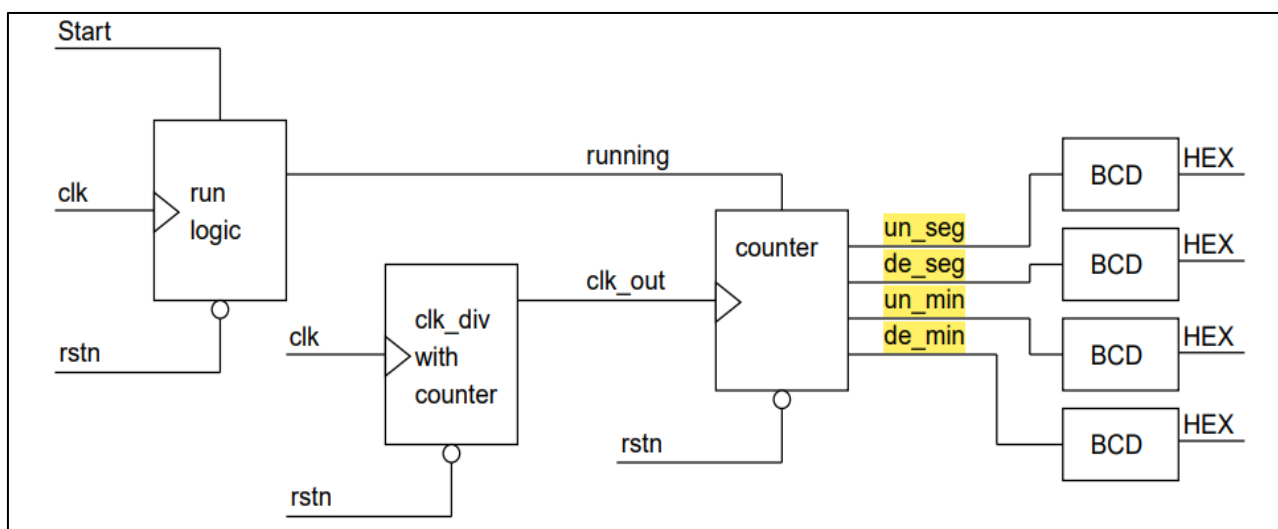
Introducción

En el presente informe se describe el plan de verificación de un proyecto de diseño digital cuyo objetivo es implementar un **reloj digital de minutos y segundos** utilizando lógica RTL codificada en SystemVerilog y sintetizada mediante la herramienta Vivado. El sistema cuenta con una interfaz de usuario simple basada en **cuatro displays de 7 segmentos**, donde se representan las unidades y decenas tanto de los segundos como de los minutos. Las señales de entrada y salida fueron definidas para controlar y visualizar el conteo de tiempo, y la funcionalidad central se basa en contadores sincronizados y un divisor de frecuencia.

Este proyecto no solo busca demostrar el dominio en diseño de sistemas digitales, sino también enfatizar la importancia de una verificación rigurosa, al asegurar que el comportamiento del circuito RTL coincida con las especificaciones funcionales establecidas. La verificación se abordará mediante simulaciones en el entorno Vivado, permitiendo validar desde el correcto funcionamiento del decodificador BCD hasta la lógica de conteo y sincronización temporal.

Marco Téorico

Los relojes digitales son sistemas electrónicos diseñados para medir y mostrar el tiempo en formato numérico, típicamente utilizando displays de 7 segmentos. A diferencia de los relojes analógicos, su funcionamiento se basa en la contabilización de pulsos de reloj, que son señales digitales periódicas generadas por un oscilador o cristal de cuarzo, las cuales sirven como referencia temporal para el sistema.



1. Contadores

Un contador digital es un circuito secuencial que incrementa o decrementa su valor en cada ciclo de reloj. En este proyecto, se emplean contadores binarios ascendentes, los cuales registran la cuenta de segundos y minutos. A través de lógica combinacional, se reinician al alcanzar un valor límite (por ejemplo, 60 segundos = 0 a 59).

2. Divisor de Frecuencia

Los sistemas digitales suelen operar a frecuencias elevadas (varios MHz), por lo que es necesario reducir esta velocidad para contar en escalas humanas (como segundos). Esto se logra mediante un divisor de frecuencia, que utiliza un contador para generar una señal de menor frecuencia a partir del reloj principal. En este caso, cada unidad de segundo se incrementa cada dos ciclos del reloj dividido, aunque el número de ciclos puede ser parametrizable.

3. Codificación BCD y Display de 7 Segmentos

Los displays de 7 segmentos permiten representar dígitos decimales del 0 al 9. Sin embargo, como la lógica interna opera en binario, es necesario usar un decodificador BCD (Binary Coded Decimal) para traducir los valores binarios de 4 bits en señales que encienden los segmentos adecuados del display.

4. Señales de Control

- **clk:** Señal de reloj del sistema.
- **rstn:** Reset asíncrono que pone el sistema en su estado inicial.
- **start_stop:** Habilita o detiene el conteo del reloj.

Estas señales permiten controlar el comportamiento del reloj, permitiendo pruebas tanto de su operación normal como de eventos límite como reinicio o conteo pausado.

5. Arquitectura del Reloj Digital

La arquitectura del sistema se basa en:

- Un módulo divisor de frecuencia.
- Un módulo principal que contiene los contadores para segundos y minutos.
- Múltiples decodificadores BCD a 7 segmentos, uno para cada dígito del reloj.

El diseño modular permite facilitar la verificación individual de cada componente, así como la integración del sistema completo.

Entrada/Salida	Nombre de la señal	Descripción
Input	clk	Señal de reloj del circuito
Input	rstn	Señal de reset asíncrona
Input	start_stop	Señal de inicio y paro del conteo
Output	hex0	Salida de display 7 segmentos para las unidades de segundo
Output	hex1	Salida de display 7 segmentos para las decenas de segundo
Output	hex2	Salida de display 7 segmentos para las unidades de minuto
Output	hex3	Salida de display 7 segmentos para las decenas de minuto

Tabla I
PINOUT DEL RELOJ DIGITAL

Desarrollo de RTL

El módulo **digital_clock** implementa la lógica de un reloj digital capaz de contar minutos y segundos, representando su salida en cuatro displays de 7 segmentos. El sistema recibe tres señales de entrada: una señal de reloj (**clk**), una de reinicio asíncrono (**rstn**) y otra de control (**start_stop**) que actúa como interruptor para iniciar o pausar el conteo. Internamente, el módulo emplea un submódulo llamado **clk_div**, cuya función es dividir la frecuencia del reloj de entrada para generar una señal más lenta (**clk_signal**), adecuada para contar el paso del tiempo humano.

El comportamiento del reloj se basa en dos registros: seconds y minutes, ambos de 6 bits, que permiten realizar un conteo de 0 a 59. A cada flanco de subida del reloj dividido, si la señal **start_stop** está activa, el sistema incrementa el conteo de segundos y, al alcanzar 59, reinicia a 0 y aumenta el valor de los minutos. Si ambos, segundos y minutos, llegan a 59, ambos se reinician. Si **start_stop** está desactivado, el valor de los contadores se mantiene constante, lo que detiene el conteo sin alterar el estado actual.

Para mostrar el tiempo en los displays, los valores de minutos y segundos se convierten a formato BCD (decimal codificado en binario), separando unidades y decenas. Estas cifras son enviadas a cuatro módulos **bcd_to_7seg**, los cuales traducen los valores BCD en patrones de encendido para los displays de 7 segmentos (hex0 a hex3). De este modo, el diseño combina lógica secuencial para el conteo de tiempo y lógica combinacional para la visualización, permitiendo una representación precisa y dinámica del reloj digital.

1. Se declaró el módulo principal del reloj digital se indicó que trabajará con una señal de reloj, una de reinicio, una de control (inicio/paro) y cuatro salidas conectadas a displays de 7 segmentos para mostrar el tiempo.

```
1 module digital_clock(  
2     input wire clk,           // Señal de reloj  
3     input wire rstn,         // Señal de reset  
4     input wire start_stop,   // Señal de activación  
5     output wire [6:0] hex0,  // Unidades de segundo  
6     output wire [6:0] hex1,  // Decenas de segundo  
7     output wire [6:0] hex2,  // Unidades de minuto  
8     output wire [6:0] hex3   // Decenas de minuto  
9 );
```

2. Se creó una señal interna (**clk_signal**) que se usará como reloj más lento para el conteo de segundos. Esta señal se generará mediante un divisor de frecuencia y se creó la instancia el módulo **clk_div**, que genera una señal de reloj de menor frecuencia a partir de la señal clk original. Esto permite que el sistema cuente segundos en lugar de trabajar a alta frecuencia.

```
wire clk_signal;           // Señal de reloj para el conteo  
  
/* Genera una señal de reloj a partir del reloj base  
y un parámetro de conteo de ciclos de reloj*/  
clk_div clk_div_1 (  
    .clk_in(clk),           // Entra la señal del reloj original  
    .rstn(rstn),           // Señal de reset  
    .clk_out(clk_signal)    // Señal de frecuencia reducida  
);
```

3. Se definieron dos registros de 6 bits que almacenan el número de segundos y minutos transcurridos. Su tamaño permite contar hasta 59, lo cual es adecuado para un reloj convencional.

```
// Registros para segundos y minutos
reg [5:0] seconds = 0; // 6 bits para contar hasta 59
reg [5:0] minutes = 0; // 6 bits para contar hasta 59
```

4. Se creó la lógica secuencial. Este bloque always define el comportamiento del reloj:
- Si rstn está en bajo, el reloj se reinicia (segundos y minutos a cero).
 - Si start_stop está en alto, el reloj cuenta:
 - Los segundos incrementan hasta 59, luego se reinician y aumentan los minutos.
 - Los minutos también se reinician si llegan a 59.
 - Si start_stop está en bajo, el valor actual de minutos y segundos se conserva (el reloj se "pausa")
5. Se creó una instancia que convierte los valores binarios de segundos y minutos en dígitos decimales (BCD). Divide en unidades y decenas para poder representar cada dígito individualmente en su display correspondiente.

```
always @(posedge clk_signal, negedge rstn) begin
    if (!rstn) begin
        seconds <= 0; // regresa el conteo a 0
        minutes <= 0;
    end else if (start_stop) begin // funciona como un enable
        if (seconds == 59) begin // hace conteo hasta llegar al último valor
            seconds <= 0; // y resetea
            if (minutes == 59) // Es necesario preguntar cual es el conteo de min
                minutes <= 0; // Si este llego al final hay que resetear tambien
            else
                minutes <= minutes + 1; // Si si no ha llegado al final aumentar 1 al conteo
        end else begin
            seconds <= seconds + 1; // Si no ha ocurrido nada de lo anterior sumar a seg
        end
    end else if (!start_stop) begin //
        seconds <= seconds; // La cuenta se debe mantener
        minutes <= minutes; // igual aqui
    end
end
```


6. Se instanciaron cuatro módulos bcd_to_7seg que convierten cada dígito BCD a una señal de 7 bits para controlar los displays. Cada uno de los hex0 a hex3 representa un dígito del reloj (de derecha a izquierda: unidades de segundo, decenas de segundo, unidades de minuto, decenas de minuto).

```
// Decodificadores BCD a 7 segmentos
bcd_to_7seg bcd0 (.bcd(sec_units), .seg(hex0)); // segundos unidades
bcd_to_7seg bcd1 (.bcd(sec_tens), .seg(hex1)); // segundos decenas
bcd_to_7seg bcd2 (.bcd(min_units), .seg(hex2)); // minutos unidades
bcd_to_7seg bcd3 (.bcd(min_tens), .seg(hex3)); // minutos decenas
```

7. Se realizó un módulo clk_div que genera una nueva señal de reloj (clk_out) a partir de una señal de alta frecuencia (clk_in) reduciéndola según el valor del parámetro DIVISOR. Esta técnica es común en sistemas digitales que requieren trabajar con tiempos perceptibles por humanos, como en relojes, temporizadores o animaciones. Su diseño es modular, limpio y reutilizable en distintos contextos.

```
// Conversión a dígitos BCD
wire [3:0] sec_units = seconds % 10; // el residuo de la div entre 10 la cuenta para las unidades de seg
wire [3:0] sec_tens = seconds / 10; // la división entre diez para las decenas de seg
wire [3:0] min_units = minutes % 10; // Misma logica para unidades de minutos
wire [3:0] min_tens = minutes / 10; // E igual para decenas de minutos
```

8. Se creó un módulo bcd_to_7seg el cual es un decodificador que convierte un valor BCD (Binary Coded Decimal) de 4 bits, correspondiente a un dígito decimal del 0 al 9, en la codificación necesaria para representar ese número en un display de 7 segmentos de ánodo común. A través de una estructura case dentro de un bloque combinacional (always @(*)), se asigna a la salida seg un patrón de 7 bits que enciende o apaga cada uno de los segmentos del display para mostrar el número correspondiente. Por ejemplo, si la entrada es 4'd3, la salida será 7'b0110000, que representa el número 3 en el display. Si el valor de entrada no está entre 0 y 9, se asigna 7'b1111111, lo cual apaga todos los segmentos. Este módulo es esencial para traducir la lógica interna del reloj digital (que opera en binario) a una salida visual comprensible para el usuario.

```
module bcd_to_7seg(
    input [3:0] bcd,          // Las entradas a esto serán las unidades o decenas de seg o min del 0 a 9
    output reg [6:0] seg      // La salida es un display de 7 segmentos
);
    always @(*) begin
        case (bcd)            // Para la entrada bcd y con un display anodo común
            4'd0: seg = 7'b1000000; // 0x40 (Así se ven normalmente en la sim)
            4'd1: seg = 7'b1111001; // 0x79
            4'd2: seg = 7'b0100100; // 0x24
            4'd3: seg = 7'b0110000; // 0x30
            4'd4: seg = 7'b0011001; // 0x19
            4'd5: seg = 7'b0010010; // 0x12
            4'd6: seg = 7'b0000010; // 0x02
            4'd7: seg = 7'b1111000; // 0x78
            4'd8: seg = 7'b0000000; // 0x00
            4'd9: seg = 7'b0011000; // 0x18
            default: seg = 7'b1111111; // 0xFF (apagado)
        endcase
    end
endmodule
```

```
module clk_div(
    input wire clk_in, // Señal de reloj entrante, propia del sistema
    input wire rstn,   // Señal de reset
    output reg clk_out // Señal de reloj saliente
);
    parameter DIVISOR = 2; // Parametro division de señal
    reg [25:0] count = 0; // Tamaño para 50M (Tomando la DE115 como base)

    always @(posedge clk_in, negedge rstn) begin
        if (!rstn) begin // Cuando el reset se activa
            count <= 0; // La cuenta se regresa
            clk_out <= 0; // Y la señal de salida se pone en bajo
        end else begin // Si reset no se activa
            count <= count + 1; // Sumar uno a la cuenta
            if (count == DIVISOR/2 - 1) begin // Si llega al final
                clk_out <= ~clk_out; // Cambiamos el valor de la señal de salida
                count <= 0; // Reseteamos la cuenta
            end
        end
    end
endmodule
```


DESIGN VERIFICATION (DV)

Plan De Verificación (Breve Concepto)

Implica asegurar que el diseño cumple su funcionalidad esperada antes de ser implementado físicamente. Se realiza mediante simulaciones funcionales y de tiempo usando herramientas como y simuladores digitales, observando el comportamiento de las señales internas y salidas ante diferentes estímulos contempla pruebas unitarias para cada módulo y pruebas integradas para el sistema completo.

Observaciones RTL:

1.- “bcd to 7seg”:

- Agregar un “**unique case**” para que sea mas sencillo la parte de “**lint/coverage**”.

2.- “clk div”:

- Asegurar que el “DIVISOR” sea par. Con impares pudiera romper el 50% del duty.
- El ancho de “count” esta fijo a 26 bits y eso limita, pudiera ser “parametrizable”.

3.- “digital clock”:

- El bloque “else if (! start stop)” es redundante.
- En esta sección la división y modulo esta por 10 esto posiblemente pudiera ser costo en (**divisores/ROMs**).

Testbench

Este primer testbench genera estímulos básicos para confirmar la funcionalidad de instancias → digital clock.

1. El período del reloj es 20 nanosegundos, corresponde a una frecuencia de 50 MHz. Este valor se usará para generar la señal de reloj simulada.

```
module tb_digital_clock; // Módulo de prueba para el módulo digital_clock

    // Parámetro para definir el periodo del reloj de prueba (20 ns = 50 MHz)
    parameter CLK_PERIOD = 20; // 50 MHz => 20 ns
```

2. Señales necesarias para simular el módulo:

- **clk**: reloj del sistema.
- **rstn**: reset asíncrono.
- **start_stop**: controla el conteo del reloj.
- **hex0 a hex3**: salidas del reloj que se conectarán al display de 7 segmentos.

```
// Declaración de señales de prueba
logic clk;           // Señal de reloj
logic rstn;          // Señal de reset (activa en bajo)
logic start_stop;    // Señal para iniciar o detener el reloj
wire [6:0] hex0, hex1; // Salidas del display para segundos (unidades y decenas)
wire [6:0] hex2, hex3; // Salidas del display para minutos (unidades y decenas)
```

3. Se creó una instancia del módulo `digital_clock` y conecta las señales internas del testbench con las del DUT.

```
// Instancia del módulo bajo prueba (DUT: Device Under Test)
digital_clock uut (
    .clk(clk),           // Conecta reloj al DUT
    .rstn(rstn),          // Conecta reset al DUT
    .start_stop(start_stop), // Conecta control de inicio/parada
    .hex0(hex0),          // Conecta salida de unidades de segundo
    .hex1(hex1),          // Conecta salida de decenas de segundo
    .hex2(hex2),          // Conecta salida de unidades de minuto
    .hex3(hex3)           // Conecta salida de decenas de minuto
);
```

4. Se generó una señal de reloj cuadrada de 50 MHz alternando el valor de `clk` cada 10 ns. Activa la lógica secuencial del módulo.

```
// Bloque inicial para generar una señal de reloj de 50 MHz
initial clk = 0;           // Inicializa el reloj en 0
always # (CLK_PERIOD/2) clk = ~clk; // Invierte el reloj cada 10 ns para crear un periodo de 20 ns
```

5. Se simuló una secuencia de eventos típica:

- Inicia con reset activo.
- Libera el reset después de unos ciclos.
- Activa el conteo del reloj.
- Pausa el reloj tras cierto tiempo.
- Lo reanuda y deja que cuente un poco más.

Cada etapa, imprime mensajes con \$display para dar seguimiento en la consola

```
initial begin
    $display("Iniciando simulación.."); // Mensaje de inicio de simulación

    rstn = 0;                // Activa el reset (activo en bajo)
    start_stop = 0;          // Inicializa el control en modo detenido

    #(5 * CLK_PERIOD);       // Espera 5 ciclos de reloj (100 ns aprox.)
    rstn = 1;                // Libera el reset
    $display("Reset liberado");

    #(20 * CLK_PERIOD);       // Espera 20 ciclos de reloj antes de iniciar
    start_stop = 1;          // Activa el reloj (comienza el conteo)
    $display("Start");

    // Deja correr la simulación por suficiente tiempo para observar conteo
    #(3000 * CLK_PERIOD);     // Espera 3000 ciclos de reloj (60 µs aprox. con CLK = 50 MHz)

    // Detiene el reloj
    start_stop = 0;          // Detiene el conteo
    $display("Stop");
    #(100 * CLK_PERIOD);      // Espera un poco antes de continuar

    // Reanuda el reloj
    start_stop = 1;          // Activa nuevamente el conteo
    $display("Start de nuevo");
    #(2000 * CLK_PERIOD);     // Espera adicional para observar el comportamiento

    $finish;                 // Finaliza la simulación
end
```

Section	Main Purpose
Modulo declaration	Definition for environment simulation
Parameter CLK_PERIOD	Sets the frequency of the simulated clock signal.
Signals Tests	They emulate inputs and monitor outputs from the design.
Instantiation of the DUT	Connect the testbench to the digital_clock module.

Section	Main Purpouse
Clock generator	Simulates a periodic signal to activate sequential logic.
Input stimuli (initial)	Simulate different operating conditions to test the module.

Especificaciones

En el pinout se detallan las siguientes especificaciones:

1. El reloj mostrara la hora (segundos y minutos) cuando **la senal de start ~ stop == 1** o alto.
2. La hora se mostrará en 4 displays de 7 segmentos unidades de segundo, decenas de segundos, unidades de minutos y decenas de minutos.
3. Las salidas para el 7 segmento de las unidades de segundo, hex0, dependerá del valor decimal de la señal interna **sec units** → Tabla II.
4. La señal interna **sec - units** incrementara una unidad desde el 0 hasta el 9 cada 2 ciclos de reloj, siempre y cuando **start - stop** este en alto. La cantidad de ciclos es parametrizable, pero fue trabajada de esta forma durante el desarrollo
5. Cuando la señal interna **sec - units** está en 9, esta regresará a 0 en su siguiente incremento.
6. Las salidas para el 7 segmento de las decenas de segundo, hex1, dependerá del valor decimal de la señal interna **sec – tens** → Tabla II.
7. **sec - tens** incrementara una unidad desde el 0 hasta el 5 cada que la señal **sec - units** regrese a 0 después de estar en 9.
8. Cuando la señal interna **sec - tens** está en 5, esta regresará a 0 en su siguiente incremento.
9. Las salidas para el 7 segmento de las unidades de minuto, hex2, dependerán del valor decimal de la señal interna min units → Tabla II.

10. La señal interna **min - units** incrementara en una unidad desde el 0 hasta el 9 cada que la señal **sec - tens** regrese a 0 después de estar en 5.
11. Cuando la señal interna **min - units** está en 9, esta regresará a 0 en su siguiente incremento.
12. Las salidas para el 7 segmento de las decenas de minuto, hex3, dependerán del valor decimal de la señal interna **min tens**, tal y como se muestra en la Tabla II.
13. La señal interna **min - tens** incrementara en una unidad desde el 0 hasta el 6 cada que la señal **min - units** regrese a 0 después de estar en 9.
14. Cuando la señal interna **min - tens** está en 6, esta regresará a 0 en su siguiente incremento.
15. Reset asíncrono, **rstn**, se activará con el flanco de bajada, y hará que la salida de todos los displays sea '1000000, correspondiente a un 0 de sus señales internas correspondientes.

TEST CASE & CHECKS. (UNA VEZ COMPLETA DEBE ACTUALIZARSE IMAGEN)

Status	Responsable	Atributo	Objetivo	Descripcion	Comentarios
COMPLETED	Alonso Emmanuel Lopez	TestCase	Reset asincrono	Verificar que al activar rstn en bajo, el reloj se reinicia.	Todos los contadores vuelven a cero.
IN PROGRESS	Alonso Emmanuel Lopez	Check	Desbordamiento de min_units a Incremento de min_tens	Verificar que hex3 incremente cada vez que hex2 pasa de 9 a 0	hex3 cuente de 0 a 6.
IN PROGRESS	Jose Antonio Garcia	TestCase	Desbordamiento de sec_units a Incremento de sec_tens	Asegurar que sec_tens se incremente cuando sec_units pasa de 9 a 0.	hex1 se incremente de 0 a 5 cada 10 ciclos de hex0.
IN PROGRESS	Jose Antonio Garcia	Check	Desbordamiento de sec_tens a Incremento de min_units	Verificar que después de 60 segundos, hex2 incremente.	hex2 sube 1 cada 60 segundos (cuando hex1 pasa de 5 a 0).
IN PROGRESS	Luis Alberto Castañeda	TestCase	Paro de reloj	Confirmar que el conteo de detiene	Todos los valores de hex permanecen constantes.
COMPLETED	Cesar Eduardo Inda Cenicerros	TestCase	Conformar los valores de tabla II	Verificar que los datos de la tabla II. Los cuales son la señal interna y la salida de 7 segmentos.	Los valores de La señal se deben mostrar en orden a las salidas de en el test de verificación.
COMPLETED	Cesar Eduardo Inda Cenicerros	Check	Conteo en Unidades de Segundo	Verificar que hex0 se incremente cada 2 ciclos de clk_out cuando start_stop = 1.	hex0 cuente de 0 a 9 en binario codificado a 7 segmentos.

Atributo 1: Conformar los valores de tabla II

Valor de señal interna	Salida del Display 7 segmentos
0	1000000
1	1111001
2	0100100
3	0110000
4	0011001
5	0010010
6	0000010
7	1111000
8	0000000
9	0011000

Tabla II

SALIDA DE LOS DISPLAYS DE 7 SEGMENTOS SEGÚN EL VALOR DECIMAL DE SU SALIDA INTERNA CORRESPONDIENTE.

Aquí se valida que **módulo bcd_to_7seg** genere correctamente los patrones de salida para un display de 7 segmentos, recorre los valores BCD del 0 al 9 asignándolos a la entrada, tras la **propagación** compara la salida generada con el patrón esperado mediante aserciones. Si el valor coincide con el definido en la tabla, la prueba continúa; de lo contrario, se reporta un error indicando el valor incorrecto.

```

initial clk = 0;
always #(CLK_PERIOD/2) clk = ~clk;

// Función para obtener patrón esperado según dígito BCD
function [6:0] expected_seg(input [3:0] bcd);
    case (bcd)
        4'd0: expected_seg = 7'b1000000;
        4'd1: expected_seg = 7'b1111001;
        4'd2: expected_seg = 7'b0100100;
        4'd3: expected_seg = 7'b0110000;
        4'd4: expected_seg = 7'b0011001;
        4'd5: expected_seg = 7'b0010010;
        4'd6: expected_seg = 7'b0000010;
        4'd7: expected_seg = 7'b1111000;
        4'd8: expected_seg = 7'b0000000;
        4'd9: expected_seg = 7'b0011000;
        default: expected_seg = 7'b1111111; // apagado
    endcase
endfunction

```



```

initial begin
    $display("Iniciando verificación del reloj digital con valores BCD y salida 7 segmentos.");
    // Reset inicial
    rstn = 0; start_stop = 0;
    #(5*CLK_PERIOD);
    rstn = 1;
    // Activar reloj
    start_stop = 1;
    // Esperamos suficiente tiempo para cubrir 0:00 hasta al menos 1 minuto
    // Cada incremento de segundo depende del divisor en clk_div (ajústalo si es necesario para simulación rápida)
    repeat (70) begin // 70 incrementos para probar múltiples valores
        #(200); // Tiempo simbólico para permitir cambio (ajusta según divisor en DUT)

        // Aserciones para hex0, hex1, hex2, hex3
        assert(hex0 == expected_seg(uut.sec_units))
            else $error("Error en hex0 (segundos unidad) esperado=%b obtenido=%b", expected_seg(uut.sec_units), hex0);

        assert(hex1 == expected_seg(uut.sec_tens))
            else $error("Error en hex1 (segundos decena) esperado=%b obtenido=%b", expected_seg(uut.sec_tens), hex1);

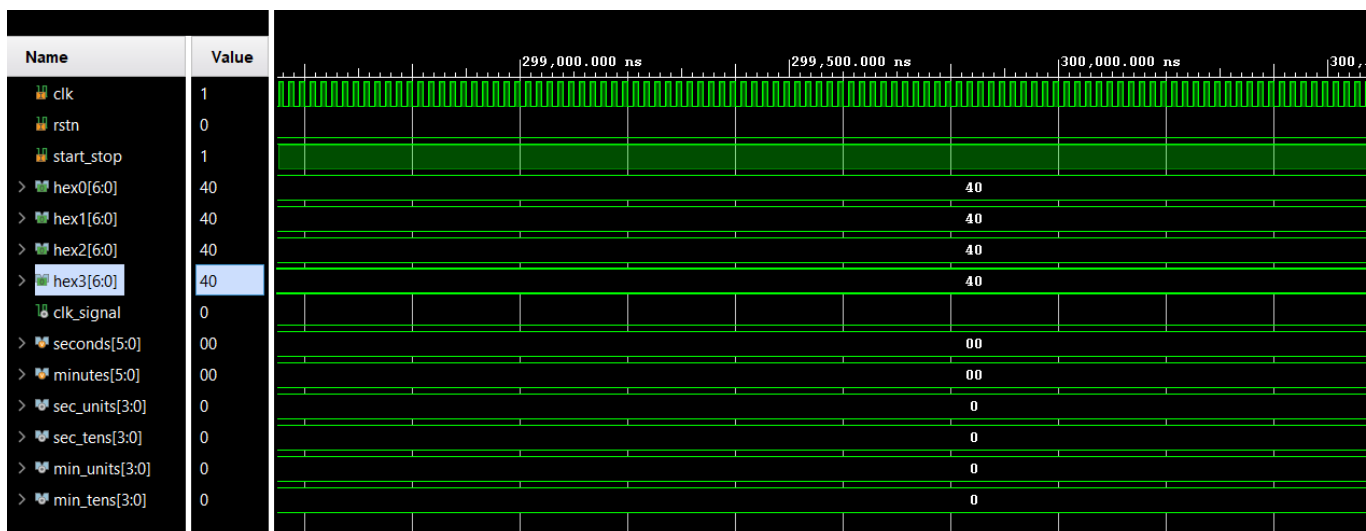
        assert(hex2 == expected_seg(uut.min_units))
            else $error("Error en hex2 (minutos unidad) esperado=%b obtenido=%b", expected_seg(uut.min_units), hex2);

        assert(hex3 == expected_seg(uut.min_tens))
            else $error("Error en hex3 (minutos decena) esperado=%b obtenido=%b", expected_seg(uut.min_tens), hex3);
    end

    $display("Verificación completada sin errores (si no se mostraron mensajes de fallo).");
    $finish;
end

```

Waverform (Vivado):



Atributo 2: Reset Asíncrono

Este Testbench verifica que al activar **rstn** en bajo durante la simulación, el reloj se reinicia inmediatamente sin utilizar un flanco de reloj. Se espera que los contadores internos de segundos y minutos regresen a cero, y en consecuencia, las salidas hex0, hex1, hex2 y hex3 correspondan al patrón del número 0 en display de 7 segmentos.

1. Iniciar la simulación con **rstn = 1 (inactivo)** y **start_stop = 1** (habilita el conteo).
2. Esperar unos ciclos para que el reloj comience a contar.
3. Forzar **rstn = 0** en un instante aleatorio (no sincronizado con el reloj). Los contadores se reinician a "0".
4. Confirmar que las salidas hex0 a hex3 muestren el patrón 7'b1000000, correspondiente a dígito cero (0 en display).
5. Antes de activar **rstn**, las salidas "**hex***" muestran un número distinto de cero, porque el reloj ya había contado unos segundos.
6. Justo al aplicar **rstn = 0**, en cualquier punto del ciclo, las salidas cambian inmediatamente a 7'b1000000 en todos los dígitos.
7. No es necesario esperar un flanco de reloj: la lógica se reinicia de inmediato.
8. El waveform debe mostrar una caída en **rstn** y simultáneamente los registros seconds y minutes volviendo a 0, y los hex0 a hex3 cambiando a "0".

```
// Generador de reloj
initial clk = 0;
always #(CLK_PERIOD/2) clk = ~clk;

// Test principal
initial begin
    $display("Iniciando test de reset asíncrono...");

    // Paso 1: iniciar normalmente
    rstn = 1;
    start_stop = 1;
    #(200); // Esperar un tiempo para que el reloj cuente

    // Paso 2: aplicar reset en un punto no sincronizado
    $display("Aplicando reset asíncrono...");
    rstn = 0;
    #(10); // Esperar un poco (simulación)

    // Comprobación visual con asserts
    assert(hex0 == 7'b1000000) else $error("Fallo en hex0 (esperado 0)");
    assert(hex1 == 7'b1000000) else $error("Fallo en hex1 (esperado 0)");
    assert(hex2 == 7'b1000000) else $error("Fallo en hex2 (esperado 0)");
    assert(hex3 == 7'b1000000) else $error("Fallo en hex3 (esperado 0)");

    $display("Reset asincrono verificado correctamente.");
    $finish;
end
```

TestCase_ValoresTablall (Cesar Inda) → “module tb_bcd_to_7seg_tableII”

Se implemento una función “**automatic**” para la tabla de los **ánodos**.

```
`timescale 1ns/1ps

// Aquí esperamos comprobar que el decodificador
//bcd_to_7seg produce exactamente los patrones 0-9 de la tabla, en orden.

module tb_bcd_to_7seg_tableII;

    logic [3:0] bcd;
    wire [6:0] seg;

    // DUT
    bcd_to_7seg dut(.bcd(bcd), .seg(seg));

    // Tabla II (ánodo común, activos en bajo)
    function automatic [6:0] expected_seg(input [3:0] v);
        case (v)
            4'd0: expected_seg = 7'b1000000;
            4'd1: expected_seg = 7'b1111001;
            4'd2: expected_seg = 7'b0100100;
            4'd3: expected_seg = 7'b0110000;
            4'd4: expected_seg = 7'b0011001;
            4'd5: expected_seg = 7'b0010010;
            4'd6: expected_seg = 7'b0000010;
            4'd7: expected_seg = 7'b1111000;
            4'd8: expected_seg = 7'b0000000;
            4'd9: expected_seg = 7'b0011000;
            default: expected_seg = 7'b1111111;
        endcase
    endfunction
endmodule
```

En esta parte se estan probando los “**Testcase**” para propagar “**assert**”.

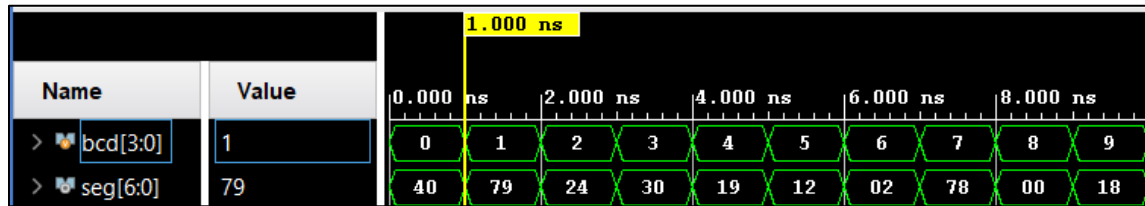
```
initial begin
    $display("=== Testcase Tabla II: barrer 0..9 en orden ===");
    // Barrido en ORDEN 0->9
    foreach (bcd[i]) begin end // evita warnings
    for (int i = 0; i <= 9; i++) begin
        bcd = i[3:0];
        #1; // delta para propagar
        assert (seg == expected_seg(bcd))
            else $error("Tabla II fallo: bcd=%0d seg=%b esperado=%b", bcd, seg, expected_seg(bcd))
        end
        $display("Tabla II verificada (0..9 en orden).");
        $finish;
    end
endmodule
```

Simulación:

En consola se puede apreciar lo siguiente:

```
# run 1000ns
=== Testcase Tabla II: barrer 0..9 en orden ===
Tabla II verificada (0..9 en orden).
$finish called at time : 10 ns : File "D:/Vivado/Team9_ProjectVerif_Team13_Module5/dv_desi
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_bcd_to_7seg_tableII_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:11 . Memory (MB): peak = 122
```

En la forma de onda se puede ver lo siguiente:



Consideración futura: Podría agregar un barrido a de 10 a 15 para que el decodificador responda con un apagado → (7'b1111111).

Check Cesar Inda) → “tb_check_hex0_every2cycles”

```
`timescale 1ns/1ps

module tb_check_hex0_every2cycles;

    // Reloj del sistema (al DUT le entra este clk; internamente divide a clk_out)
    parameter int CLK_PERIOD = 20; // 50 MHz
    logic clk, rstn, start_stop;
    wire [6:0] hex0, hex1, hex2, hex3;

    // DUT
    digital_clock uut (
        .clk(clk),
        .rstn(rstn),
        .start_stop(start_stop),
        .hex0(hex0), .hex1(hex1), .hex2(hex2), .hex3(hex3)
    );

    //Generación del reloj
    initial clk = 0;
    always #(CLK_PERIOD/2) clk = ~clk;

    //Decodificador inverso: 7-seg (ánodo común, activos en bajo)
    function automatic [3:0] seg_to_bcd(input [6:0] s);
        case (s)
            7'b1000000: seg_to_bcd = 4'd0;
            7'b1111001: seg_to_bcd = 4'd1;
            7'b0100100: seg_to_bcd = 4'd2;
            7'b0110000: seg_to_bcd = 4'd3;
            7'b0011001: seg_to_bcd = 4'd4;
            7'b0010010: seg_to_bcd = 4'd5;
            7'b0000010: seg_to_bcd = 4'd6;
            7'b1111000: seg_to_bcd = 4'd7;
            7'b0000000: seg_to_bcd = 4'd8;
            7'b0011000: seg_to_bcd = 4'd9;
            default: seg_to_bcd = 4'hF; //inválido
        endcase
    endfunction
endfunction
```

Se realiza el initial begin para sección de reset y arranque.

```
///! ----- CHECK: hex0 incrementa cada 2 ciclos de clk_out (clk_signal) -----  
///! * Interpretamos "ciclo" de clk_out como media oscilación (flanco a flanco).  
///! Por eso contamos ambos flancos (posedge/negedge): 2 flancos = 1 periodo.  
///! Tras 2 flancos, hex0 debe INCREMENTAR (0..9), entre medias debe mantenerse.  
  
int edge_cnt;           // cuenta de flancos de clk_out (clk_signal)  
int last_digit;         // dígito previo (0..9)  
int curr_digit;         // dígito actual (0..9)  
int increments_seen;    // cuántos incrementos válidos vimos  
  
initial begin  
    // Reset y arranque  
    rstn = 0; start_stop = 0;  
    repeat (5) @(posedge clk);  
    rstn = 1; start_stop = 1;  
  
    // Esperar a que hex0 muestre un patrón válido y tomar referencia  
    wait (seg_to_bcd(hex0) != 4'hF);  
    last_digit = seg_to_bcd(hex0);  
    edge_cnt = 0;  
    increments_seen = 0;
```

Se definen los incrementos de dígitos en base a flancos.

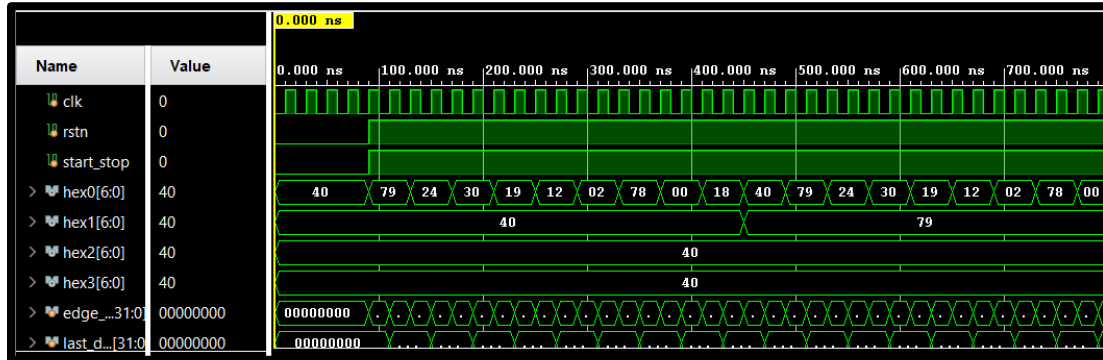
```
// Monitorear el reloj dividido interno del DUT  
// (clk_out del divisor se llama clk_signal dentro de digital_clock)  
repeat (40) begin // 40 flancos -> 20 periodos -> >=10 incrementos  
    @(posedge uut.clk_signal or negedge uut.clk_signal);  
    edge_cnt++;  
  
    curr_digit = seg_to_bcd(hex0);  
    assert (curr_digit != 4'hF)  
        else $fatal("Patrón 7-seg inválido en hex0=%b", hex0);  
  
    if ((edge_cnt % 2) == 0) begin  
        // Cada 2 flancos (1 periodo completo) debe INCREMENTAR 0..9  
        int expected = (last_digit + 1) % 10;  
        assert (curr_digit == expected)  
            else $error("hex0 NO incrementó cada 2 ciclos: esp=%0d obt=%0d (hex0=%b, flancos=%0d)",  
                expected, curr_digit, hex0, edge_cnt);  
        last_digit = curr_digit;  
        increments_seen++;  
    end  
    else begin  
        // Entre medias debe mantenerse  
        assert (curr_digit == last_digit)  
            else $error("hex0 cambió ANTES de completar 2 ciclos: prev=%0d ahora=%0d (flancos=%0d)",  
                last_digit, curr_digit, edge_cnt);  
    end  
end  
  
// Debe haber recorrido al menos 0..9  
assert (increments_seen >= 10)  
    else $error("Se esperaban >=10 incrementos (0..9). Vistos=%0d", increments_seen);  
  
$display("CHECK OK: hex0 incrementa cada 2 ciclos de clk_out y recorre 0..9 en 7-seg.");  
$finish;  
end  
  
endmodule
```

Simulación:

En consola se puede apreciar lo siguiente:

```
# run 1000ns
CHECK OK: hex0 incrementa cada 2 ciclos de clk_out y recorre 0..9 en 7-seg.
$finish called at time : 870 ns : File "D:/Vivado/Team9_ProjectVerif_Team13_Module5/dv_designve
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_check_hex0_every2cycles_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:07 . Memory (MB): peak = 2257.547
```

En la forma de onda se puede ver lo siguiente:



Desbordamiento de “min_units” a Incremento de “min_tens”.

Este bloque de verificación se confirma que el módulo del reloj digital, el cambio de valor en las unidades de minuto (salida hex2) de 9 a 0 provoca el incremento correcto del dígito de decenas de minuto (salida hex3). Se espera que, cada vez que se produzca la transición de 9 → 0 en hex2, el valor de hex3 aumente en 1, reflejando el comportamiento de un contador BCD para minutos.

El bloque de verificación descrito:

- Inicializa adecuadamente el entorno del testbench (reset, activación del contador).
- Monitorea y convierte las salidas de los dígitos del contador de minutos en valores numéricos.
- Verifica de forma automática que la transición de las unidades de minuto desde 9 a 0 produce el incremento en el dígito de las decenas.
- Permite evidenciar, mediante mensajes en consola, el correcto funcionamiento del sistema ante los casos de borde en el conteo.

El “testbench” valido el comportamiento del módulo del reloj digital, sincronizando correctamente del contador de minutos y su representación en displays de 7 segmentos.

```
module tb_incremento_min;

    // Señales internas
    reg clk = 0;
    reg rstn = 0;
    reg start_stop = 0;

    wire [6:0] hex0, hex1, hex2, hex3;

    // Instancia del DUT
    digital_clock uut (
        .clk(clk),
        .rstn(rstn),
        .start_stop(start_stop),
        .hex0(hex0),
        .hex1(hex1),
        .hex2(hex2),
        .hex3(hex3)
    );

    // Generador de reloj (50 MHz → periodo de 20ns)
    always #10 clk = ~clk;

    // Variables internas para verificar valores
    integer total_seconds = 0;
```

```
// Tareas auxiliares
task print_time;
    $display("Tiempo simulado: %0t ns -> %0d:%0d", $time, uut.minutes, uut.seconds);
endtask

initial begin
    $display("Iniciando testbench del reloj digital...");

    // Reset activo en bajo
    rstn = 0;
    start_stop = 0;
    #50;

    // Liberamos reset
    rstn = 1;
    #50;

    // Activamos el conteo
    start_stop = 1;

    // Simulamos suficiente tiempo para cubrir 60 minutos (60 * 60 segundos)
    // Esperamos a cada pulso del `clk_signal` → depende del divisor
    // Suponiendo que el divisor da 1Hz, entonces 1 pulso por segundo
    // En ese caso, el test dura 3600 "segundos"
    repeat (3600) begin
        @(posedge uut.clk_signal); // Esperamos a cada pulso de 1 Hz
        total_seconds = total_seconds + 1;
    end
```



```
// Monitoreamos los momentos clave
if (uut.seconds % 10 == 0 && uut.seconds <= 59)
    print_time();

// Verificamos desbordamientos de unidades a decenas
if (uut.seconds == 0 && uut.minutes > 0 && total_seconds != 0)
    $display(">>> Segundo volvió a 0, minuto incrementado a %0d", uut.minutes);

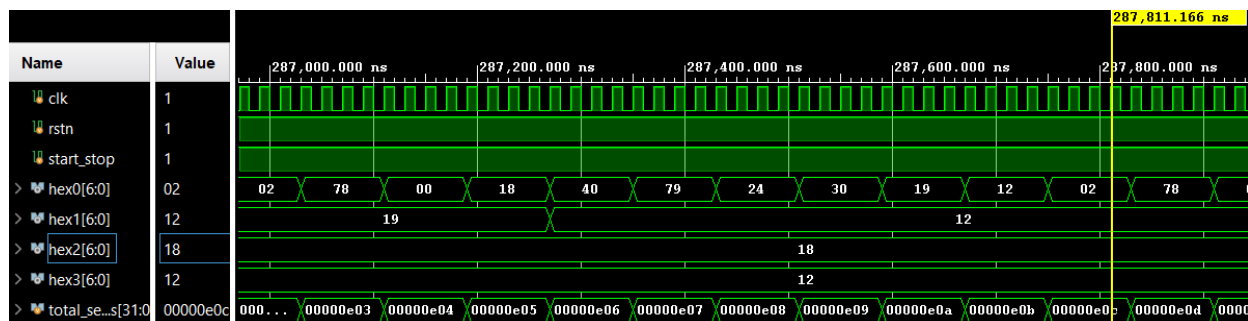
// Verificación final: debe reiniciar en 00:00 después de 59:59
if (uut.minutes == 0 && uut.seconds == 0 && total_seconds == 3600)
    $display(">>> ✔ El reloj reinició correctamente después de 59:59");

end

$display("Testbench finalizado. Tiempo simulado total: %0d segundos", total_seconds);
$stop;

end
endmodule
```

Para propósito de este “**Testbench**” fue posible observar la siguiente forma de onda en el “**Waverform**” de la simulación.



Cover Group → tb_coverage_hex0

1. “**cp_digits**” comprueba que hex0 presentó **todos los dígitos 0..9**.
2. “**cp_inc**” verifica la **secuencia 0→1→...→9→0**.
3. **cp_inc_every2** verifica que el incremento sucede tras **dos muestras consecutivas** del mismo valor, es decir, **cada 2 flancos** de clk_out (tu clk_signal).

```
// ===== Covergroup mínimo (1 coverpoint) =====
// Objetivo: ver TODOS los dígitos 0..9 en hex0 => 100% cobertura.
covergroup cg_hex0;
    option.per_instance = 1;
    cp_digits: coverpoint digit_sampled iff (en_sampled) {
        bins digits[] = {[0:9]};
    }
endgroup

cg_hex0 cov = new();

// Muestreo manual en posedge del reloj dividido (paso natural del conteo)
always @(posedge uut.clk_signal) begin
    digit_sampled = seg_to_bcd(hex0);
    en_sampled = (rstn && start_stop && (digit_sampled != 4'hF));
    cov.sample();
end
```

Se estuvieron realizando algunos cambios, pero solo se pudo observar el 66.7% en la cobertura.

Date	08/15/25 19:56:44 Mountain Standard Time (Mexico)
User	tiori
Version	Vivado Simulator Coverage Report 2024.2.0
Command Line	C:/Xilinx/Vivado/2024.2/bin/unwrapped/win64.o/xcrng.exe -report_format html -dir D:/Vivado/Team9_ProjectVerif_Team13_Module5/Verification_DigitalClock/Verification_DigitalClock.sim/sim_1/behav/xsim/xsim.covdb -report_dir D:/Vivado/Team9_ProjectVerif_Team13_Module5/Verification_DigitalClock/Verification_DigitalClock.sim/sim_1/behav/xsim/xcrng_report
Number of Tests	1

Total Groups Coverage Summary	
Score	Inst Score
66.6667	66.6667

GitHub

https://github.com/CeicGitHub/Fundamentals Of Digital Design FPGA 5/tree/Project_VerificationDigitalClk

Conclusiones / Issues

Cesar Eduardo Inda Cenicerros

Este proyecto me transmitió mucho conocimiento en cuanto a conocer más profundo la parte de verificación, el proceso que existe para validar un diseño, considero que la dinámica de trabajo no fue la mejor, fue complicado estar en comunicación con el equipo encargado de realizar el RTL en continuidad que nosotros realizábamos la verificación y esa parte entorpeció bastante el flujo de trabajo, posiblemente algo bueno en practicas futuras seria dividir la sección de diseño y verificación pero en el mismo equipo internamente, es decir considerar un equipo de 4 personas, donde 2 de ellos realicen el RTL y los otros 2 realicen la verificación, para poder tener un flujo de trabajo más ordenado.

Alonso Emmanuel Lopez Macias

El proyecto a mi parecer fue un tanto complejo por que requirió ser mayormente minucioso en la parte de verificar muchas validaciones, “testbench”, “asserts”, y “covergroups” eso también sin descartar el hecho de analizar mas a detalle las formas de onda de cada simulación y justificar su comportamiento, comprendiendo que si el diseño tenía errores, nosotros tendríamos que documentar esas partes y validar aquellas que pudieran ser mayormente validadas.